

Payment-Integrity Claims Reviewer

Rules detect, AI explains, a human decides — with a measurable, auditable review loop.

A hands-on walkthrough: set it up, click through it, turn on the AI explanations, run the evaluation, and poke at the internals. No prior context needed.

The one idea to hold onto: rules detect, AI explains, a human decides. Transparent, deterministic rules decide *which* claims get flagged; the LLM only writes a plain-English reason for a flag (it never flags, clears, or ranks anything); a human reviewer makes the actual call. Everything below is built around keeping those three jobs separate.

All data is **synthetic** with authored ground-truth labels — no real claims, no PHI.

1. What you're running

A small FastAPI web app that:

1. **Generates** synthetic medical claims with known issues (duplicates, unbundling, out-of-network mismatches), clean claims, and tricky "near-miss" look-alikes.
2. **Flags** the problematic ones with inspectable rules — each flag records the rule id, the exact fields that triggered it, and a confidence score.
3. **Explains** each flag in plain English with Claude (optional — needs an API key).
4. **Routes** flags to a reviewer queue where you approve / dismiss / escalate.
5. **Reports** flagged volume, reviewer outcomes, and an **estimated** dollars-identified figure (clearly labeled an estimate, with its assumptions shown).
6. **Measures** itself: detector precision/recall/F1 vs the ground-truth labels, plus a check that the explanations don't invent reasons.

2. Prerequisites

- **Python 3.11+** (`python --version`).
- **Git** (optional, only if you want the commit history).
- **An Anthropic API key** — *optional*. Detection, the queue, the dashboard, and the eval all work without one. The key is only needed to generate the LLM explanations.
- `make` is convenient but **not required** — every command has a plain `python -m ...` equivalent (see the [command reference](#)).

3. Setup (one time)

From the project folder (`payment-integrity-reviewer`):

Windows (PowerShell):

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1    # your prompt now shows (.venv)
pip install -r requirements.txt
```

macOS / Linux:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Once the virtual environment is **activated**, `python` (and `make`, which calls `python`) points at the project's isolated install. If you don't want to activate it, prefix make commands with the interpreter path, e.g. `make reset PY=.venv/Scripts/python`.

4. The 5-minute tour

Step 1 — create data

```
make reset
```

This wipes the database, generates the labeled synthetic claims, and runs the rules engine. You'll see something like `Detected 34 flags (8 duplicate / 18 unbundling / 8 oon_mismatch)`.

Step 2 — start the app

```
make run
```

Open `http://localhost:8000`. (Stop the server later with `Ctrl+C`.)

Step 3 — read the dashboard

You land on the **Dashboard**: flags raised, claim lines flagged, pending/approved counts, and **Estimated dollars identified** — shown twice: *Identified* (across all flagged claims) and *Confirmed* (only reviewer-approved flags). It carries an explicit **"ESTIMATE — not recovered dollars"** badge, and the per-issue assumptions are printed right there.

Step 4 — open a flag

Click **Review queue** → **Review** → on any flag. The detail page shows three things: - **Why it was flagged** — the rule id and the *exact triggering fields* (the auditable reason). - **Plain-English explanation** — the LLM's rationale (or a "run

`make explain` " note if you haven't generated them yet). - **The claim line** — the underlying claim fields. (Note: the ground-truth label is deliberately hidden — a real reviewer wouldn't get the answer key.)

Step 5 — make a decision

Use **Approve / Dismiss / Escalate** (optionally add your name + a note). You're returned to the queue; the decision is stored as a label. Go back to the **Dashboard** — an approved flag now shows under **Confirmed** dollars, and the outcome counts update.

That's the whole loop: **a rule flagged it, you saw exactly why, and you — the human — decided.**

5. Turning on the AI explanations

Detection never needs a key. To see the plain-English rationales:

1. Copy the example env file and add your key: `bash # Windows: copy .env.example .env # macOS/Linux: cp .env.example .env` Edit `.env` and set `ANTHROPIC_API_KEY=sk-ant-...` (You can also change `ANTHROPIC_MODEL` ; it defaults to `claude-opus-4-8`.)
2. Generate the explanations: `bash make explain`
3. Reload a flag's detail page — the explanation now appears, with the **model and prompt version** recorded beneath it.

Each explanation is grounded strictly in the triggering rule + fields, and the system prompt forbids the model from recommending an outcome. If a call fails (rate limit, etc.), that flag is skipped and reported; just re-run `make explain` to fill it in.

6. Running the evaluation

```
make eval
```

Because the synthetic claims carry ground-truth labels, detection quality is measurable:

```
DETECTOR (overall)    precision 1.00  recall 1.00  F1 1.00
duplicate             precision 1.00  recall 1.00  F1 1.00
unbundling            precision 1.00  recall 1.00  F1 1.00
oon_mismatch          precision 1.00  recall 1.00  F1 1.00
false positives: []   false negatives: []
near-misses correctly not flagged: 17/17
EXPLANATION FAITHFULNESS  n/a - no explanations generated. Run `make explain`.
```

How to read it: scores are 1.00 because the seed is rule-aligned by construction — that's expected and stated honestly. The number that actually means something is **near-misses correctly not flagged: 17/17**: those are claims engineered to *look* like issues but be legitimate (a same-day repeat with a modifier, an authorized out-of-network visit). Skipping them is what "precision" really tests here. With explanations generated (and a key present), the faithfulness line scores how many explanations stay grounded in the rule + fields, judged both by a deterministic check and an LLM-as-judge.

7. Resetting to a clean state

Anytime the data gets messy (you've approved a bunch of flags, etc.):

```
make reset
```

Wipes the DB, re-seeds, and re-detects. It does **not** re-run `make explain` (that needs the key and a network call), so re-generate explanations afterward if you want them.

8. How it works under the hood

The pipeline, and where to look in the code:

Stage	Module	What it does
Seed	app/seed.py	Deterministic generator → labeled claims in SQLite
Reference data	app/reference.py	CPT prices, unbundling panels, distinct-service modifiers
Detect	app/detect.py	The three transparent rules → <code>flag</code> rows
Explain	app/explain.py	LLM rationale grounded in the rule + fields
Review + ROI	app/dashboard.py , app/roi.py	Queue, decisions, savings math
Web	app/main.py , app/templates/	Routes + HTML
Eval	app/eval.py	Precision/recall/F1 + faithfulness

Data model (SQLite, [app/models.py](#)): `claim` (one row per claim line, with the ground-truth label used only by the eval), `flag` (rule id, triggering fields, confidence, and the LLM explanation columns), `decision` (each approve/dismiss/escalate, append-only).

The **three rules** and their near-misses: - **Duplicate** — same member+provider+CPT+date, submitted again, *unless* a distinct-service modifier (76/59/...) makes the repeat legitimate. - **Unbundling** — a panel's component codes billed separately instead of the one panel code, *unless* every component carries a distinct-service modifier. - **Out-of-network mismatch** — an out-of-network provider paid as in-network with no authorization/emergency on file.

For *why* each design choice was made (including why the LLM doesn't detect), read [DECISIONS.md](#).

9. Command reference

Task	With <code>make</code>	Without <code>make</code>
Install deps	<code>make install</code>	<code>python -m pip install -r requirements.txt</code>
Seed claims	<code>make seed</code>	<code>python -m app.seed</code>
Run detection	<code>make detect</code>	<code>python -m app.detect</code>
Generate explanations	<code>make explain</code>	<code>python -m app.explain</code>
Start the app	<code>make run</code>	<code>python -m uvicorn app.main:app --reload</code>
Run the eval	<code>make eval</code>	<code>python -m app.eval</code>
Run tests	<code>make test</code>	<code>python -m pytest</code>
Clean reset	<code>make reset</code>	<code>python -m app.seed && python -m app.detect</code>

10. Things to try

- **Change an ROI assumption.** Edit the percentages in `app/roi.py` (e.g. the OON pricing delta), refresh the dashboard — the figure and the surfaced assumptions both update. Keep `data/roi_assumptions.md` in sync.
- **Add or tweak a rule.** Adjust `app/detect.py`, then `make detect && make eval` and watch precision/recall move. (Try removing the modifier check and see the near-misses become false positives.)
- **Add a claim.** Extend a scenario in `app/seed.py`, `make reset`, and find it in the queue.
- **Swap the model.** Set `ANTHROPIC_MODEL=claude-sonnet-4-6` in `.env` and re-run `make explain` — the recorded model on each explanation changes.

11. Troubleshooting

Symptom	Fix
<code>No module named app</code>	Run commands from the project root (the folder with <code>app/</code>).
<code>make: command not found</code> (Windows)	Use the <code>python -m ...</code> column above, or install make.
Dashboard/queue say "No data yet"	Run <code>make reset</code> to seed + detect, then refresh.
Explanation panel says "run <code>make explain</code> "	Expected until you add a key to <code>.env</code> and run it.
<code>Skipped: ANTHROPIC_API_KEY is not set</code>	Add your key to <code>.env</code> (see §5). Detection still works without it.
Port 8000 in use	<code>python -m uvicorn app.main:app --reload --port 8001</code>
<code>wrong python</code> after activating venv	Re-open the terminal and re-activate, or use <code>PY=.venv/Scripts/python</code> .
<code>make explain</code> fails with <code>CERTIFICATE_VERIFY_FAILED</code>	A corporate/AV root CA isn't in Python's bundle. The app already uses <code>truststore</code> (the OS trust store) automatically; if it persists, ensure <code>pip install truststore</code> ran and the CA is in your OS store.

12. Responsible-AI recap

- **Synthetic only** — no real claims or PHI.
- **Detection is transparent** — every flag traces to a concrete rule + fields, no opaque model verdict.
- **AI explains, it does not decide** — the LLM can't flag or clear a claim, and is constrained from inventing reasons (and that constraint is measured in the eval).
- **A human decides** — and every decision is logged as a label.
- **The dollars figure is an estimate** — with documented assumptions, never presented as recovered money.

Where to go next

- [DEMO.md](#) — the tight ~90-second demo script.
- [DECISIONS.md](#) — why it's built this way (the "why not let the LLM detect?" answer).
- [EVAL.md](#) — what the eval measures and the thresholds.
- [README.md](#) — overview and the Path to Production.